Personalized Agentic AI Coworkers

DailyTechUpgradeAgent and nextLevelPersonalCoachForCareerLevel

Meron Tesfandrias, MS Software Engineering (Data Science), San José State University

These two agents share one memory. The first helps me learn something new every day. The second prepares me for my next career milestone, a Summer 2027 software engineering internship, and for possible entrepreneurial ventures. What the coach finds I am weak at feeds into what the daily agent has me read.

1. Agent goals

DailyTechUpgradeAgent

Purpose: make me a better technical learner every day by finding the three most useful technical articles and turning them into a 15–20 minute reading session.

Expected outcomes:

- Three relevant, high-quality articles each day from authoritative sources, each with title, date, source and URL.
- A summary, key insights and takeaways for each, plus how it helps my knowledge and career goals.
- One new concept per article and a recall question, so I learn something new and can check that I did.
- Recommendations that adapt to my interests, my ratings, what I have already read and the gaps found by the coach.

nextLevelPersonalCoachForCareerLevel

Purpose: prepare me for my next milestone by finding opportunities, measuring my readiness and helping me improve.

Expected outcomes:

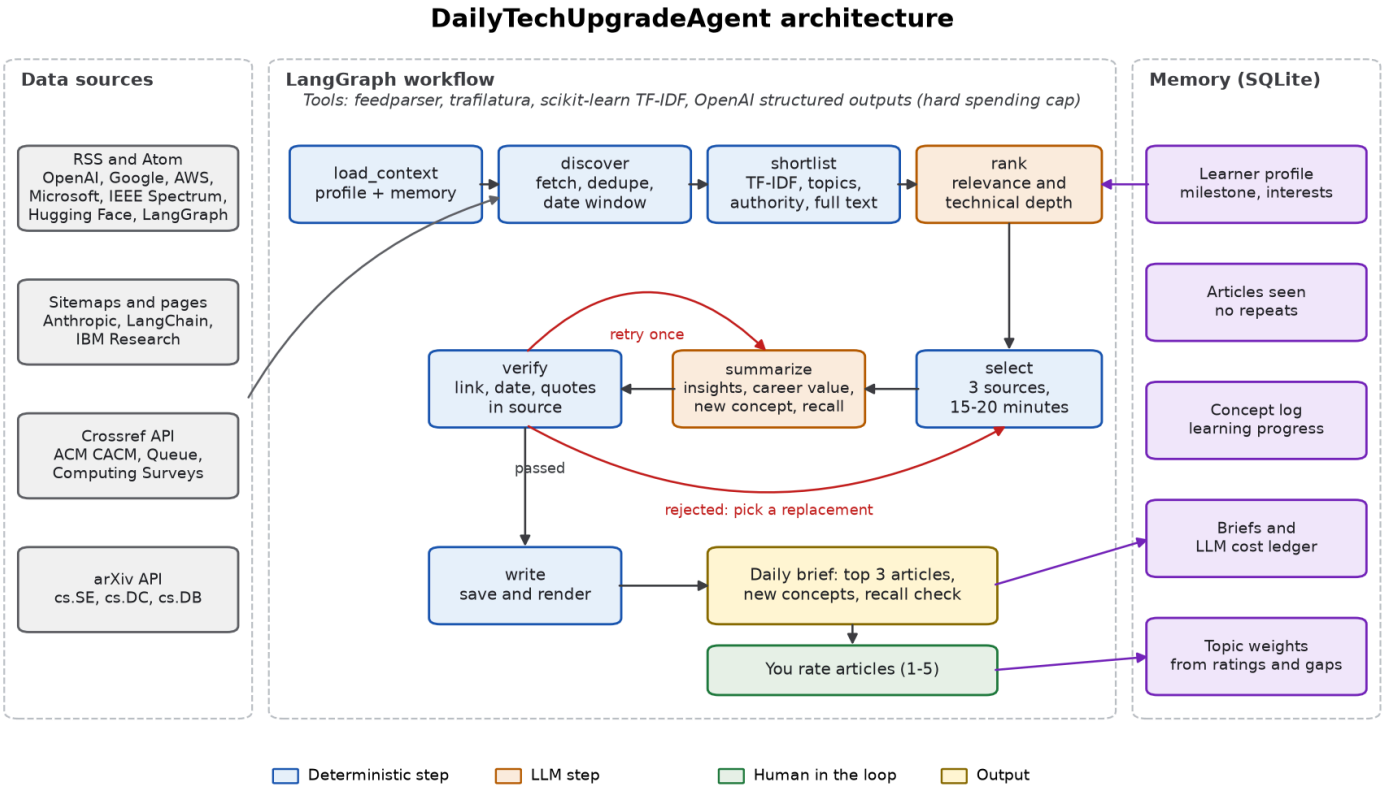
- Relevant internships, jobs, research leads and entrepreneurial signals, each scored for fit.
- Analysis of job descriptions, technical blogs and research papers to work out which skills matter.
- Personalized technical questions, coding exercises and interview scenarios.

- An interactive session that questions me, evaluates my answers and challenges them with follow-ups.
- A Personalized Career Readiness Report with strengths, gaps, resources, a preparation plan and progress over time.

2. Architecture

Both agents are LangGraph state machines. Fixed code handles anything that can be checked exactly, such as dates, links, scoring formulas and memory. The LLM handles judgment and writing.

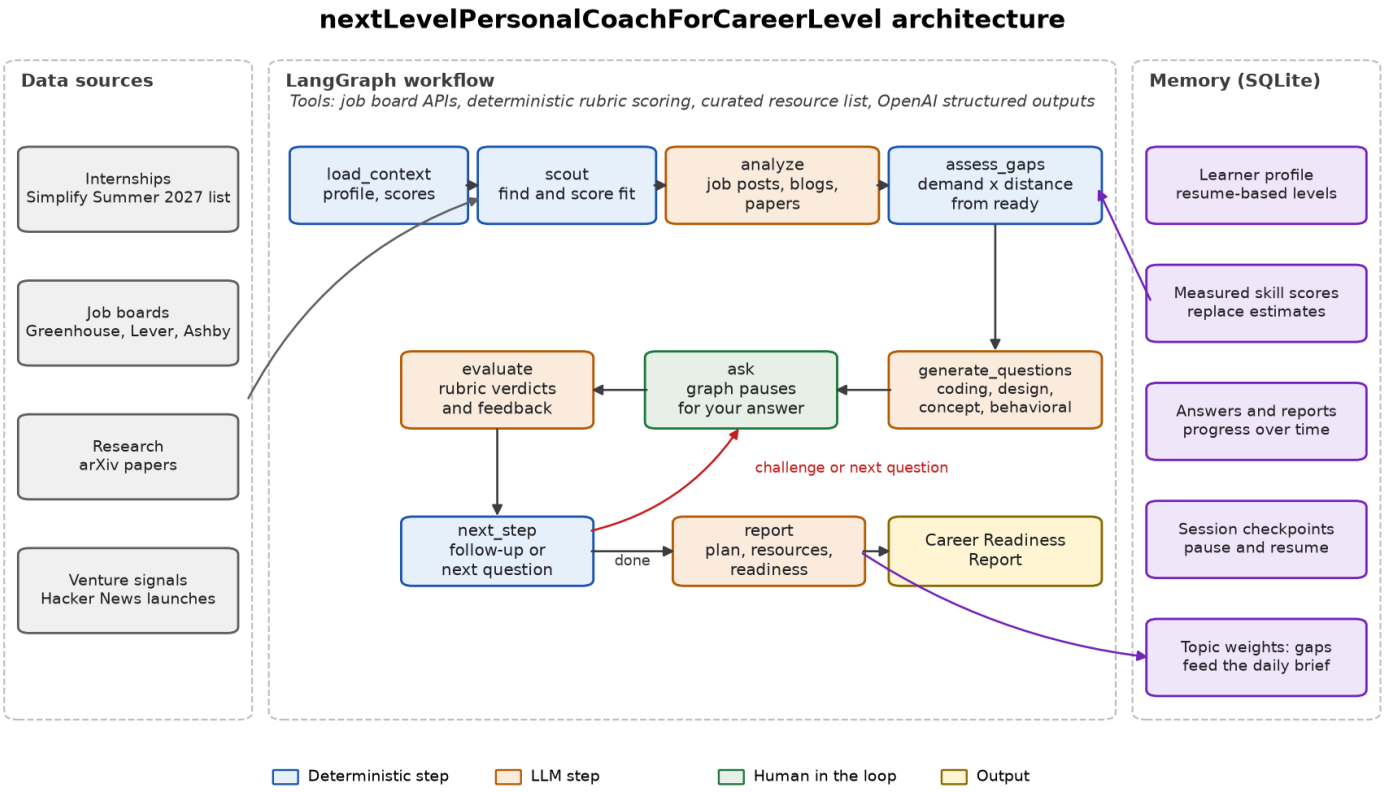
DailyTechUpgradeAgent



Workflow: the agent loads my profile and memory, gathers articles from 15+ feeds and APIs, removes anything old or already seen, and pre-ranks with TF-IDF, topic weights, source authority and freshness. Only the top 12 reach the LLM, which keeps costs low. The LLM scores relevance and technical depth, and a selector picks three articles that fit the time budget. Sources stay distinct when possible, and one source may appear twice only when that is the only way to reach three.

Verification: each summary must quote the article word for word to back up every insight. Fixed code confirms each quote appears in the source, the link works, the date is in range and the concept is new to me. A failed summary gets one retry, and then the article is replaced.

nextLevelPersonalCoachForCareerLevel



Workflow: the coach finds opportunities, analyzes job posts together with recent blogs and papers, and ranks skills by market demand times my distance from ready. It writes questions for the biggest gaps. The graph then pauses using LangGraph's `interrupt`, and a saved checkpoint means a session can be stopped and resumed.

Scoring: the LLM marks each rubric item yes, partly or no. Code turns those verdicts into the score, so the number always matches the written feedback. Measured scores replace the estimates that came from my resume, and the report states how much of the readiness score is measured.

Grounded recommendations: the coach can only recommend from a checked resource list or my own briefs, so it cannot invent a link.

Shared memory and the learning loop

Direction	What flows
Coach to daily agent	The biggest skill gaps raise those topics' weights in the daily ranking.
Daily agent to coach	Recent brief articles feed market analysis and become recommended resources.
Me to daily agent	Article ratings move topic weights up or down.

Safety and cost

- Web pages and answers are treated as untrusted data, so instructions inside them are ignored. The evaluation tests this with a prompt injection.
- A hard spending cap stops API calls before they exceed the budget.
- Simulated sessions are flagged and cannot change real progress.

3. Working prototype

- **Command line:** `python -m coworkers daily`, `coach`, `replay`, `rate`, `opportunities`, `status`.
- **Web app:** `streamlit run app.py` provides the brief, an interactive coaching tab and a progress dashboard.
- **Daily automation:** `scripts/run_daily.ps1` can be scheduled with Windows Task Scheduler.
- **Tests:** `python -m pytest tests` covers both graphs, verification, scoring and the spending cap.

Built with: LangGraph, the OpenAI API with structured outputs, SQLite, scikit-learn, Streamlit, feedparser and trafilatura.

Requirement coverage

Requirement	Where it is met
Three high-quality articles daily	`rank`, `select` and `verify` nodes in the daily graph
Summaries, insights, takeaways	`summarize` node
Titles, dates, sources, URLs	Brief format and References section
Knowledge and career contribution	`knowledge_contribution` and `career_contribution` fields
Personalized to interests and progress	Topic weights, ratings, seen articles, concept log, coach gaps
15–20 minute brief	Reading-time budget in the selector
Learn something new every day	New-concept check and recall questions
Internships, jobs, research, entrepreneurial	`scout` node
Analyze job posts, blogs, papers, materials	`analyze` node
Questions, exercises, scenarios	`generate_questions` node
Interactive questioning and challenge	`ask`, `evaluate` and `next_step` loop
Strengths, gaps, improvement areas	Rubric evaluation and report
Resources and action plans	`report` node and resource list
Track progress	Skill scores, readiness history, progress dashboard

4. Sample results

Replayed briefs are generated later, but they only use articles published on or before that date and are labeled as replays.

Daily technical recommendations

All generated briefs:

- `samples/briefs/2026-10-27.md`
- `samples/briefs/2026-10-28.md`
- `samples/briefs/2026-10-29.md`
- `samples/briefs/2026-10-30.md`
- `samples/briefs/2026-10-31.md`
- `samples/briefs/2026-11-01.md`

The most recent one is below.

Daily Technical Learning Brief: 2026-11-01

Reading session: about 18 minutes. **Concepts logged so far:** 18. Chosen from 186 new articles in the last 7 days.

Today's new concepts

- **h-Majority consensus trade-off:** In synchronous h-Majority, each node repeatedly samples h random peers and updates to the most frequent sampled opinion. The article shows consensus time improves with h and can become O(log n) when the leading opinion is sufficiently ahead, and it proves a near-matching lower bound for a broad class of update rules.
- **Bedrock ZDR mode:** When Zero Data Retention (ZDR) is enabled, prompts and responses are processed without being written to durable storage by AWS or shared with model providers. Use this to satisfy stringent data-governance and privacy requirements while still routing inference through Amazon Bedrock in the selected AWS Region.
- **Browser handoff:** When a website requires login or 2FA, use the browser handoff so the user signs in directly. This prevents the password from entering model messages or traces, and the agent resumes after control is handed back.

1. Nearly-Optimal Time-Sample Trade-off in Consensus Dynamics

Source: arXiv | **Published:** 2026-10-29 | **Read:** 5 min **URL:** https://arxiv.org/abs/2610.40562v1

The article studies synchronous h-Majority consensus dynamics in which each node repeatedly samples h random peers and adopts the most frequent sampled opinion. It shows that increasing the sample size h improves the consensus time, but also derives nearly-optimal bounds describing how the trade-off scales with network size n and the initial support p_max. The paper also provides faster convergence guarantees when the leading opinion has a sufficiently large advantage, and it proves a lower bound showing the result is close to best possible (up to polylog factors) for a broad class of update rules.

Key insights

- In h-Majority, consensus time depends on both the sample size h and the initial concentration p_max, scaling roughly like min{1/(h^2 p_max), sqrt(n)/h} within polylog factors. *("the process reaches consensus with high probability within \widetilde{O}\left(1+\min\left{\frac{1}{\min{h^2p_{max},1}},\frac{\sqrt{n}}{h}\right}\right) rounds")*
- There is a special fast regime: if the initially most popular opinion has enough lead over all others, nodes converge in O(log n) rounds with rate controlled by min{h^2 p_max,1}. *("all nodes adopt this opinion with high probability within O(\log n/\min{h^2p_{max},1}) rounds")*
- For update rules that treat opinions symmetrically and only choose between a sampled opinion or the node's current opinion, the paper proves a matching lower bound (up to polylog factors), indicating near-optimality of h-Majority's trade-off. *("We establish a matching lower bound up to polylogarithmic factors for any fixed update rule that treats all opinions symmetrically and that selects either a sampled opinion or the node's current opinion.")*

Takeaway: When designing or analyzing randomized consensus dynamics, model each node's update as sampling h peers; then expect convergence time to improve with larger h, especially when an opinion starts with a strong initial advantage (large p_max).

Adds to your knowledge: You now have concrete, quantitative consensus-time bounds for the synchronous h-Majority rule, including a nearly-optimal upper bound valid for every h≥3 and every initial configuration, plus a complementary lower bound for a broad class of symmetric update rules.

Helps your career goal: This strengthens your distributed-systems fundamentals around randomized consensus performance and trade-offs (sample size vs. time), which is directly relevant to backend/distributed backend interviews and internship project planning.

Where to focus: Focus on the three main results: the general consensus-time bound (the \widetilde{O}(...) expression), the fast-case bound when the leading opinion has sufficient advantage (the O(log n/...) expression), and the lower-bound statement describing near-optimality for symmetric update rules.

Summarized from the abstract only, because the full text was not available.

2. Configure the ChatGPT desktop app with Amazon Bedrock in AWS GovCloud (US)

Source: AWS Machine Learning | **Published:** 2026-10-30 | **Read:** 5 min **URL:** https://aws.amazon.com/blogs/machine-learning/configure-the-chatgpt-desktop-app-with-amazon-bedrock-in-aws-govcloud-us/

This article explains how to configure the ChatGPT desktop app to route model inference through Amazon Bedrock in AWS GovCloud (US), specifically us-gov-west-1. It emphasizes a security-first, controlled local deployment using AWS identities/endpoints/Regional controls plus desktop-side policy files. It also covers two authentication pathways (Bedrock API key or AWS SDK credential chain) and a Zero Data Retention option for data residency and governance requirements.

Key insights

- Use API-mode sign-in plus a managed device policy and network restrictions to constrain the desktop app to an approved Bedrock endpoint. *("This post walks through a controlled local deployment of the ChatGPT desktop app that sends model inference to Amazon Bedrock in AWS GovCloud (US-West). The configuration uses API-mode sign-in, a managed device policy, a named AWS profile, and network restrictions.")*
- Authentication precedence matters: if AWS_BEARER_TOKEN_BEDROCK is present, the app uses that bearer token instead of profile-based authentication. *("The app first checks for AWS_BEARER_TOKEN_BEDROCK in its environment. For desktop use, this variable can be supplied through %USERPROFILE%.codex.env on Windows or ~/.codex/.env on Linux and macOS.")*
- To meet strict privacy/data-governance needs, the article supports Zero Data Retention (ZDR), which processes prompts/responses without writing to durable storage or sharing with model providers. *("Zero data retention (ZDR) is also supported. In this configurable mode, prompts and responses are processed without being written to durable storage by AWS or shared with model providers.")*

Takeaway: Implement a controlled deployment by pairing Bedrock routing (us-gov-west-1) with explicit desktop policy/config files and restricted network access, and validate which authentication source takes precedence.

Adds to your knowledge: Adds practical cloud-integration details for sending ChatGPT/Codex inference through Amazon Bedrock in AWS GovCloud (US), including the exact authentication precedence behavior and the governance-oriented ZDR capability.

Helps your career goal: Strengthens your internship-aligned backend/cloud readiness by giving you a concrete pattern for secure model-inference architecture: IAM identity/endpoint scoping, region confinement, and privacy controls.

Where to focus: Read the sections: “Solution overview” (two ways: ChatGPT app vs Codex), “Step 1: Set up authentication” (bearer token vs credential chain/SSO), “Step 2: Set up ChatGPT desktop app

configuration files" (requirements.toml vs config.toml), and "Troubleshooting" (common misconfiguration causes).

3. Managed Deep Agents v0.10: Browser Use and Async Subagents

Source: LangChain | **Published:** 2026-10-29 | **Read:** 5 min **URL:** https://www.langchain.com/blog/managed-deep-agents-browser-use-and-asynchronous-subagents

Managed Deep Agents v0.10 adds two core capabilities: browser use for website interaction without APIs, and async subagents for long-running background work. Browser use includes a "handoff" flow that keeps credentials out of model messages and traces. Async subagents run on separate threads and report back so the orchestrator keeps responding without blocking.

Key insights

- Use browser_handoff to keep user credentials out of model messages/traces when a site requires login. *("With browser_handoff, the agent hands the window to you instead, so your password never ends up in the model's messages or traces.")*
- Run long tasks with async subagents so the orchestrator agent doesn't block while the subagent runs in the background. *("you can run subagents asynchronously, without blocking the orchestrator agent, by adding async to a subagent definition:")*
- Ensure thread replies (including scheduled runs and finished subagents) land in the original Slack thread, and cancel them with a Stop button. *("Replies go back to the Slack thread a request started in, including replies from a scheduled run or from a subagent that finished later.")*

Takeaway: When you need website actions or long background work, combine browser_handoff (for credential safety) with async subagents (to keep your main agent responsive), while relying on thread-scoped replies and Stop to manage execution.

Adds to your knowledge: This extends your agent orchestration toolkit with (1) a credential-safe browser handoff mechanism, (2) truly non-blocking async subagent execution, and (3) deterministic Slack-thread routing plus cancellation controls for distributed task workflows.

Helps your career goal: These patterns map directly to internship-relevant backend/distributed design concerns: secure external-system integration, concurrency without blocking, and reliable observability/control via thread-aware task management.

Where to focus: Read closely: "Give your agent a browser" (especially browser_handoff), "Run subagents in the background" (the async subagent model + notification behavior), and "Thread origins and stopping runs" (runtime.thread + Stop).

Recall check

Answer these from memory after reading.

1. According to the article, within what type of bound (in terms of n, h, and p_max) does h-Majority reach consensus for any initial configuration?
2. In the article, what authentication source takes precedence when AWS_BEARER_TOKEN_BEDROCK is set for the ChatGPT desktop app?
3. What does browser_handoff do to protect credentials, and how does it affect what the agent sees after the user signs in?

Answers

1. With high probability within \widetilde{O}\left(1+\min\left{\frac{1}{\min{h^2p_{max},1}},\frac{\sqrt{n}}{h}\right}\right) rounds.
2. The app first checks for AWS_BEARER_TOKEN_BEDROCK, and when present the bearer token takes precedence over profile-based authentication.
3. It hands the browser window to the user so the password never ends up in the model's messages or traces; once you sign in and hand control back, the agent checks the page to see whether the login worked and continues.

References

1. Nearly-Optimal Time-Sample Trade-off in Consensus Dynamics. arXiv, 2026-10-29. https://arxiv.org/abs/2610.40562v1
2. Configure the ChatGPT desktop app with Amazon Bedrock in AWS GovCloud (US). AWS Machine Learning, 2026-10-30. https://aws.amazon.com/blogs/machine-learning/configure-the-chatgpt-desktop-app-with-amazon-bedrock-in-aws-govcloud-us/
3. Managed Deep Agents v0.10: Browser Use and Async Subagents. LangChain, 2026-10-29. https://www.langchain.com/blog/managed-deep-agents-browser-use-and-asynchronous-subagents

Interactive career coaching session

The answers in this session are my own.

All real sessions:

- `samples/coaching/report-20261101-1937-6097.md`

The most recent one is below.

Personalized Career Readiness Report: 2026-11-01

Milestone: Summer 2027 software engineering internship

Readiness: 32.2/100. 37.8% of that weight comes from answers measured in coaching sessions. The rest is estimated from the resume.

You are not close enough for a comfortable Summer 2027 internship target yet, but the gap is still tractable because most of the scored evidence is from a small number of coding prompts. The main risk is breadth: DSA is only partially demonstrated, while Python, system design, and distributed systems currently have no measured evidence.

Technical assessment

Skill	Type	Follow-up	Score
dsa	coding	no	25.0
dsa	coding	yes	0.0
python	coding	no	0.0
system-design	system_design	no	0.0
distributed-systems	scenario	no	0.0

Strengths

- The DSA answer shows partial understanding of the problem structure: a single pass and a one-deletion flag are both relevant to this class of question.
- You used a strict comparison (`l < c`), which matches the strictly increasing requirement.
- The assessment suggests you are starting from the right direction on array monotonicity problems, even though the solution is incomplete.

Knowledge gaps

- You do not yet have a correct algorithm for deciding which element to delete when the sequence breaks.
- You are missing the prefix/suffix run logic needed for an O(n) solution to the DSA problem.
- You did not explain how to compute the maximum stable batch length or justify correctness.
- You provided no evidence yet for Python coding ability.
- You provided no evidence yet for system design or distributed-systems reasoning.

Skill priorities

Skill	Market demand (1-5)	Level	Evidence	Priority
dsa	5	0.12	measured	4.375

Skill	Market demand (1-5)	Level	Evidence	Priority
python	4	0.0	measured	4.0
system-design	4	0.0	measured	4.0
distributed-systems	4	0.0	measured	4.0
ai-engineering	3	0.25	resume estimate	2.25
api-design	4	0.5	resume estimate	2.0
cloud-aws	4	0.6	resume estimate	1.6
behavioral	3	0.5	resume estimate	1.5
testing-ci	3	0.55	resume estimate	1.35
networking	2	0.35	resume estimate	1.3
databases-sql	2	0.4	resume estimate	1.2
java	2	0.4	resume estimate	1.2
concurrency	3	0.65	resume estimate	1.05
cpp	2	0.65	resume estimate	0.7

Preparation plan

Action	Skill	Due	Done when
Solve 8-10 one-deletion array problems and write the decision rule for deletion in each ca	dsa	14 days	You can explain the delete-or-keep choice for each problem without referring to notes, and
Practice prefix/suffix reasoning on monotonic-array problems until you can derive the O(n)	dsa	10 days	You can derive the prefix/suffix run lengths and use them to solve a fresh problem in one
Write 5 short Python implementations of array/string utilities, focusing on clean loops, i	python	12 days	Each solution runs correctly on your own test cases and you can explain the time complexit
Read one introductory system design resource and summarize the core concepts: load balanci	system-design	18 days	You can sketch a basic design for a simple service and name the main bottlenecks and trade
Study one distributed-systems resource and focus on consistency, replication, and fault to	distributed-systems	21 days	You can explain the difference between availability, consistency, and partition tolerance

Recommended resources

- NeetCode roadmap
- Tech Interview Handbook
- System Design 101 (ByteByteGo)
- The System Design Primer
- Python asyncio docs

Opportunities

Internships

Title	Org	Location	Posted	Fit
Software Engineer Intern (Summer 2027)	Figma	San Francisco, CA • New York, NY	2026-10-27	0.96
Software Engineering Intern (2027)	Databricks	Bellevue, Washington; Mountain View, California; San Francisco, California; Vancouver, Can	2026-10-23	0.96
Software Engineer, Internship - Infrastructure	Palantir	Palo Alto, CA	2025-08-19	0.92
Software Engineer Intern	VSP Vision	Remote in USA	2026-10-28	0.9
Software Engineer Intern - Technology Innovation	VSP Vision	Remote in USA	2026-10-28	0.9
Software Engineer Intern	VSP Vision	Remote in USA	2026-10-28	0.9
Software Engineer Intern	VSP Vision	Remote in USA	2026-10-28	0.9
Software Engineer Intern - First Play	LinkedIn	Mountain View, CA	2026-10-28	0.9

Entry-level jobs

Title	Org	Location	Posted	Fit
Software Engineer, Early Career — Immediate Start	Stripe	San Francisco, Seattle, New York	2026-10-29	0.9
Software Engineer, New Grad	Stripe	San Francisco, Seattle, New York	2026-10-29	0.9

Title	Org	Location	Posted	Fit
Software Engineer - New Grad (2027 Start)	Databricks	Bellevue, Washington; Mountain View, California; San Francisco, California; Vancouver, Can	2026-10-27	0.843
Software Engineer, New Grad - Infrastructure	Palantir	Palo Alto, CA	2025-08-19	0.82

Research leads: none found in this run.

Entrepreneurial signals

Title	Org	Location	Posted	Fit
Show HN: OmniDiff – Opensource, super fast, content aware Git difftool, in Rust	Hacker News		2026-11-01	0.6
Show HN: I built a multiplayer game where you're an aerial fighter cat	Hacker News		2026-10-30	0.6
Show HN: I Turned DOOM into a Transformer	Hacker News		2026-10-29	0.6

Research angle: Use the open-source ai-agent and observability briefs as a research direction: compare how recent systems instrument agent memory, evidence, and intent, then map those ideas onto a small tutor or coding-assistant tool.

Venture angle: Build a small developer tool for your interests in learning and math: a study/problem-review app that turns coding mistakes into spaced repetition cards and tracks proof of understanding on algorithm problems, inspired by the content-aware diff and agent-tool signals.

Market analysis

Themes: Backend/infrastructure engineering with scalability & reliability expectations; API/platform development (including agent-tool interfaces via OpenAPI/MCP); Cloud-based data/AI infrastructure and distributed systems (often AWS-centric); Agent/AI engineering and observability/verification for production agent workflows

Based on: Software Engineer Intern (Summer 2027) at Figma; Software Engineering Intern (2027) at Databricks; Software Engineer, Internship - Infrastructure at Palantir; Software Engineer, Early Career — Immediate Start at Stripe; Software Engineer, New Grad at Stripe; Software Engineer - New Grad (2027 Start) at Databricks

Fed into the daily brief: software architecture

Session transcript

Question 1: dsa (coding)

You are given a list of sensor timestamps from a fusion pipeline, each timestamp is an integer and the list may contain duplicates because the same reading can be retried. A batch is considered "stable" if its timestamps can be arranged into a strictly increasing contiguous sequence after removing at most one reading from the batch. Write pseudocode or Python for a function that returns the maximum length of a stable batch you can obtain from the entire list by removing at most one element anywhere. Aim for O(n) time and explain the core idea clearly. Example: [3, 4, 8, 5, 6, 7] -> 5 because removing 8 yields [3, 4, 5, 6, 7].

Answer:

iterate through the batches, keep track of the value l and c where l is the previous value and c is the current. start with l at index 0 and c at index 2. also keep track of the number of elements passed with variable count. iterate through the list with the given starting l and c checking to make sure l < c, if so increment count and move on. if l >= c, then we have to remove a value so remove value c, do not increment count and set a flag bool to true showing that we have used our 1 removal. the flag should be checked before we remove a value and be initlized to false. if you ever try to remove a value and flag is true then it fails and we cannot make the batch valid, if not and it iterates through its valid and the answer is count

Score: 25.0

Feedback: Your answer is too vague to justify correctness. A strong solution needs to explain how to precompute increasing runs on the left and right, then check whether deleting one element can connect them into a longer strictly increasing segment. You should also state how the function handles empty lists, one element, already increasing arrays, and cases where no deletion improves the result.

- partly: Derives an O(n) solution rather than checking all removals or all subarrays. (The answer describes a single left-to-right pass with a flag, which suggests linear time, but it never clearly states the complexity or shows how the full batch length is computed correctly.)
- partly: Correctly handles duplicates by requiring strictly increasing values, not non-decreasing values. (It uses the condition `l < c`, so it does require strict increase. However, it does not explain how duplicates are handled beyond treating `l >= c` as a failure/removal case.)
- no: Explains how to combine prefix and suffix increasing runs to evaluate the effect of deleting one element. (There is no prefix/suffix decomposition or merging of runs. The answer only describes a greedy local deletion rule.)
- no: Returns the correct answer on edge cases such as empty input, one element, already strictly increasing input, and arrays where no deletion helps. (No edge cases are addressed.

The answer does not explain what happens for empty input, a single element, already sorted input, or cases where one deletion cannot fix the batch.)

Follow-up: dsa (coding)

How would you compute, in O(n), the best length after deleting each possible element using prefix and suffix information, and how would you handle the boundary checks when joining the two sides?

Answer:

skip

Score: 0.0

Feedback: Skipped, so this counts as a gap. Study the outline, then try it again.

- no: Defines prefix/suffix increasing run lengths or an equivalent linear-time precomputation. (Not answered.)
- no: Explains the exact condition for joining left and right parts after one deletion using strict inequality. (Not answered.)
- no: Addresses at least one edge case correctly, such as empty input or an already strictly increasing array. (Not answered.)

Question 2: python (coding)

Your React/Web Worker probability calculator will now send Monte Carlo summaries to a Python backend for post-processing. Write a Python function `summarize_trials(trials)` where `trials` is an iterable of dictionaries like `{"player": "A", "success": True, "points": 27}`. The function should return a dictionary mapping each player to a summary object with: total trials, success rate, mean points over all trials, and mean points over successful trials only. You may assume the input can be large, so avoid storing all rows in memory. Include a short explanation of how your code stays memory-efficient.

Answer:

skip

Score: 0.0

Feedback: Skipped, so this counts as a gap. Study the outline, then try it again.

- no: Uses a single pass over the iterable or otherwise avoids materializing the entire dataset in memory. (Not answered.)
- no: Computes all four requested metrics correctly for each player. (Not answered.)

- no: Handles players with zero successful trials without crashing; defines a sensible value for the successful-trials mean. (Not answered.)
- no: Uses idiomatic Python data structures and clear variable names, with correct treatment of missing players in the output. (Not answered.)

Question 3: system-design (system_design)

Design the backend for the candidate's serverless NBA prop probability API so it can support a big traffic spike on game nights while staying fast and inexpensive. Assume the current stack uses AWS Lambda, API Gateway, S3, and Terraform. Walk through the request path from client to compute to storage, how you would cache results, how you would reduce cold-start impact, what happens when upstream data changes close to game time, and which metrics you would alert on. Be specific about tradeoffs and justify any AWS services you add.

Answer:

skip

Score: 0.0

Feedback: Skipped, so this counts as a gap. Study the outline, then try it again.

- no: Describes a complete request path with at least one compute layer and one persistence/cache layer, not just a list of services. (Not answered.)
- no: Explains a caching strategy that accounts for both repeated identical requests and freshness when game or player data changes. (Not answered.)
- no: Addresses cold starts, traffic spikes, and failure modes with concrete mitigation ideas such as pre-warming, queueing, or fallback responses. (Not answered.)
- no: Includes production monitoring signals such as latency, error rate, throttles, cache hit rate, and downstream freshness/staleness metrics. (Not answered.)
- no: Discusses tradeoffs between cost, latency, and correctness rather than giving a single one-size-fits-all design. (Not answered.)

Question 4: distributed-systems (scenario)

Your multithreaded multi-sensor Kalman filter is being split into separate services: one per sensor ingestion source and one central fusion service. Describe how you would design the message flow so the fusion service can maintain a correct-enough real-time state when messages are duplicated, arrive out of order, or are replayed after a restart. Explain what metadata you would put on each message, how the fusion service would detect duplicates, how it would deal with late messages, and what state it would persist to recover safely after a crash.

Answer:

skip

Score: 0.0

Feedback: Skipped, so this counts as a gap. Study the outline, then try it again.

- no: Specifies message metadata needed for ordering and duplicate detection, such as sensor ID, sequence number, and timestamp or epoch. (Not answered.)
- no: Describes a recovery strategy that persists enough state to avoid double-applying updates after restart. (Not answered.)
- no: Explains how the system handles duplicates, out-of-order delivery, and late arrivals with explicit rules. (Not answered.)
- no: Mentions at-least-once delivery semantics and the implications for idempotency or deduplication. (Not answered.)
- no: Keeps the design focused on real-time correctness and clearly states any acceptable staleness or approximation. (Not answered.)

5. Evaluation

Effectiveness asks whether the agent produces something useful. **Accuracy** asks whether it is correct. **User progress** asks whether I am improving.

Dimension	DailyTechUpgradeAgent	nextLevelPersonalCoachForCareerLevel
Effectiveness	My ratings of articles, share of sessions within 15–20 minutes, shortlist changes across profiles	Grading separates strong, partial and wrong answers; bluffs and injections don't score well
Accuracy	Link, date and quote checks, plus an independent judge of each claim	Skill extraction precision and recall against hand labels, repeat-grading consistency
User progress	Concepts logged, recall questions	Readiness and skill scores across sessions

Results

Evaluation results

Run on 2026-11-01.

DailyTechUpgradeAgent

Measure	Result
Briefs (replayed)	6 (5)
Briefs with all 3 articles	6
Links live	100.0%
Dates inside the window	100.0%
Evidence quotes found in source	100.0%
Claims an independent judge supported	97.4% of 39
Summaries passing verification first time	83.3%
Articles rejected by the verifier	0
Sessions within 15-20 minutes	100.0% [18, 18, 18, 18, 18, 18]
Distinct sources used	6
Repeated articles	0
Concepts logged	18
User ratings	0 (mean n/a, useful n/a%)

Personalization test. Same 139 articles, different learner profiles.

Comparison	Shortlist overlap
SWE focus (this profile) vs ML research focus	66.7%
SWE focus (this profile) vs Security focus	58.3%

Profile	Shortlist matching its top 3 topics
SWE focus (this profile)	66.7%
ML research focus	66.7%
Security focus	66.7%

nextLevelPersonalCoachForCareerLevel

Skill extraction on 6 hand-labeled job posts written for this test: precision 0.909, recall 0.909, F1 0.909.

Grading calibration on seeded, simulated answers:

Question	Strong	Partial	Wrong	Confident bluff	Prompt injection
Explain the difference between a mutex and a semaphore, and ...	100.0	33.3	0.0	0.0	0.0
How would you design a URL shortener handling 10,000 redirec...	100.0	12.5	0.0	0.0	0.0
Write a function that returns the length of the longest subs...	100.0	16.7	0.0	0.0	0.0

- Strong > partial > wrong held for 100.0% of questions.
- The bluff scored below the strong answer on 100.0% of questions.
- The highest score a prompt injection earned was 0.0.
- Grading the same answer twice differed by 0.0 points on average.

Resource links: 100.0% of 18 are live.

User progress: 1 real session(s), 1 question(s) answered and 4 skipped. Skipped questions score 0, so they lower the measured scores.

Baseline readiness is 32.2, with 37.8% backed by measured answers. A second session is needed to show any change.

Limitations

- Progress here covers a short period, so it shows the tracking works rather than long-term improvement. Regular use over a semester would be a stronger test.
- Replayed briefs depend on what feeds still list, and sources that only keep recent items have fewer older articles.
- ACM and arXiv summaries use abstracts, and those items are labeled in the brief. ACM's site blocks scripts, so ACM links are confirmed through the Crossref registry instead of being opened directly.
- Research leads are recent papers and venture signals are launches on Hacker News. They are starting points, not open positions or funded opportunities.
- Coding answers are graded by reading them. The code is not executed, so a subtle bug can be missed.
- Skill extraction was tested on short job posts written for the test rather than a large real sample.

Cost

Total spend: **$0.4509**

Step	Calls	Cost
coach.evaluate	92	$0.2398
eval.faithfulness	51	$0.0972
daily.summarize	44	$0.0495
coach.questions	3	$0.0211
daily.rank	11	$0.0166
coach.plan	3	$0.0147
coach.analyze	3	$0.0048
eval.extraction	24	$0.0039
eval.simulated_candidate	7	$0.0028
check	4	$0.0004